Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №2

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему

«Розробка графічного редактора об’єктів»

Виконав:

студент групи ІМ-52
Соколович Марко Юрійович
Номер в списку - 23

Перевірив:

Рекечинський Д.О.

Київ 2026

Завдання:
1. Створити у середовищі розробки новий проект з ім’ям Lab2.
2. Скомпілювати проект і отримати виконуваний файл програми.
3. Перевірити роботу програми. За потреби налагодити програму.
4. Проаналізувати та прокоментувати результати та вихідний текст програми.
5. Оформити звіт.

Зберігання фігур: Статичний масив вказівників на 123 елементів (23 + 100).
Гумовий слід: Пунктирна лінія чорного кольору (23 mod 4 = 3).
Прямокутник:

- **Ввід:** Від центру до одного з кутів (23 mod 2 = 1).
- **Відображення:** Чорний контур прямокутника без заповнення (23 mod 5 = 3).

Еліпс:

- **Ввід:** За двома протилежними кутами охоплюючого прямокутника (23 mod 2 = 1).
- **Відображення:** Чорний контур із помаранчевим заповненням (23 mod 5 = 3; 23 mod 6 = 5).

Позначка поточного типу об'єкта: Відображається в заголовку вікна (23 mod 2 = 1).

Вихідний текст головного файлу:

main.cpp

```
#include <windows.h>
#include <cmath>
#include "resource.h"
#include "shape.h"

using namespace std;

#define MAX_SHAPES 123

HINSTANCE hInst;
WCHAR szTitle[] = L"Lab2";
WCHAR szWindowClass[] = L"LAB2_WINDOW_CLASS";
```

```
Shape* pcshape[MAX_SHAPES] = { nullptr };
int shapeCount = 0;

enum ShapeType { TYPE_POINT, TYPE_LINE, TYPE_RECT, TYPE_ELLIPSE };
ShapeType currentType = TYPE_RECT;
bool isDrawing = false;
long startX = 0, startY = 0;
long prevX = 0, prevY = 0;

void UpdateTitle(HWND hWnd) {
    const wchar_t* typeName = L"";
    switch (currentType) {
    case TYPE_POINT:   typeName = L"Крапка"; break;
    case TYPE_LINE:    typeName = L"Лінія"; break;
    case TYPE_RECT:    typeName = L"Прямокутник"; break;
    case TYPE_ELLIPSE: typeName = L"Еліпс"; break;
    }
    wchar_t titleBuf[128];
    wsprintfW(titleBuf, L"Lab2 - [Поточний об'єкт: %s]", typeName);
    SetWindowTextW(hWnd, titleBuf);
}

void DrawRubberTrack(HDC hdc, long x1, long y1, long x2, long y2,
ShapeType type) {
    int oldRop = SetROP2(hdc, R2_NOTXORPEN);
    HPEN hPen = CreatePen(PS_DOT, 1, RGB(0, 0, 0));
    HGDIOBJ oldPen = SelectObject(hdc, hPen);
    HGDIOBJ oldBrush = SelectObject(hdc, GetStockObject(NULL_BRUSH));

    switch (type) {
    case TYPE_POINT:
        break;
    case TYPE_LINE:
        MoveToEx(hdc, x1, y1, NULL);
        LineTo(hdc, x2, y2);
        break;
    case TYPE_RECT: {
        long dx = abs(x2 - x1);
        long dy = abs(y2 - y1);
        Rectangle(hdc, x1 - dx, y1 - dy, x1 + dx, y1 + dy);
        break;
    }
```

```
    case TYPE_ELLIPSE:
        Ellipse(hdc, x1, y1, x2, y2);
        break;
    }

    SelectObject(hdc, oldPen);
    SelectObject(hdc, oldBrush);
    DeleteObject(hPen);
    SetROP2(hdc, oldRop);
}

Shape* CreateShape(ShapeType type, long x1, long y1, long x2, long y2)
{
    Shape* s = nullptr;
    switch (type) {
    case TYPE_POINT:   s = new PointShape(); break;
    case TYPE_LINE:    s = new LineShape(); break;
    case TYPE_RECT:    s = new RectShape(); break;
    case TYPE_ELLIPSE: s = new EllipseShape(); break;
    }
    if (s) s->Set(x1, y1, x2, y2);
    return s;
}

LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM
lParam) {
    switch (message) {
    case WM_CREATE:
        UpdateTitle(hWnd);
        break;

    case WM_COMMAND: {
        int wmId = LOWORD(wParam);
        switch (wmId) {
        case IDM_SHAPE_POINT:
            currentType = TYPE_POINT;
            UpdateTitle(hWnd);
            break;
        case IDM_SHAPE_LINE:
            currentType = TYPE_LINE;
            UpdateTitle(hWnd);
            break;
        case IDM_SHAPE_RECT:
```

```
            currentType = TYPE_RECT;
            UpdateTitle(hWnd);
            break;
        case IDM_SHAPE_ELLIPSE:
            currentType = TYPE_ELLIPSE;
            UpdateTitle(hWnd);
            break;
        case IDM_ABOUT:
            MessageBoxW(hWnd, L"Лабораторна робота №2\nВаріант 23",
L"Про програму", MB_OK | MB_ICONINFORMATION);
            break;
        case IDM_EXIT:
            DestroyWindow(hWnd);
            break;
        default:
            return DefWindowProcW(hWnd, message, wParam, lParam);
        }
        break;
    }

    case WM_LBUTTONDOWN: {
        long x = LOWORD(lParam);
        long y = HIWORD(lParam);

        if (currentType == TYPE_POINT) {
            if (shapeCount < MAX_SHAPES) {
                pcshape[shapeCount++] = CreateShape(TYPE_POINT, x, y,
x, y);
                InvalidateRect(hWnd, NULL, TRUE);
            }
        } else {
            isDrawing = true;
            startX = prevX = x;
            startY = prevY = y;
            SetCapture(hWnd);
        }
        break;
    }

    case WM_MOUSEMOVE: {
        if (isDrawing) {
            long curX = LOWORD(lParam);
            long curY = HIWORD(lParam);
```

```
            HDC hdc = GetDC(hWnd);
            DrawRubberTrack(hdc, startX, startY, prevX, prevY,
currentType);
            DrawRubberTrack(hdc, startX, startY, curX, curY,
currentType);
            ReleaseDC(hWnd, hdc);

            prevX = curX;
            prevY = curY;
        }
        break;
    }

    case WM_LBUTTONUP: {
        if (isDrawing) {
            long endX = LOWORD(lParam);
            long endY = HIWORD(lParam);

            HDC hdc = GetDC(hWnd);
            DrawRubberTrack(hdc, startX, startY, prevX, prevY,
currentType);
            ReleaseDC(hWnd, hdc);

            isDrawing = false;
            ReleaseCapture();

            if (shapeCount < MAX_SHAPES) {
                pcshape[shapeCount++] = CreateShape(currentType,
startX, startY, endX, endY);
            }
            InvalidateRect(hWnd, NULL, TRUE);
        }
        break;
    }

    case WM_PAINT: {
        PAINTSTRUCT ps;
        HDC hdc = BeginPaint(hWnd, &ps);

        for (int i = 0; i < shapeCount; ++i) {
            if (pcshape[i]) {
                pcshape[i]->Show(hdc);
```

```
            }
        }

        EndPaint(hWnd, &ps);
        break;
    }

    case WM_DESTROY:
        for (int i = 0; i < shapeCount; ++i) {
            delete pcshape[i];
            pcshape[i] = nullptr;
        }
        PostQuitMessage(0);
        break;

    default:
        return DefWindowProcW(hWnd, message, wParam, lParam);
    }
    return 0;
}

int APIENTRY wWinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
LPWSTR lpCmdLine, int nCmdShow) {
    hInst = hInstance;

    WNDCLASSEXW wcex = {};
    wcex.cbSize = sizeof(WNDCLASSEX);
    wcex.style = CS_HREDRAW | CS_VREDRAW;
    wcex.lpfnWndProc = WndProc;
    wcex.hInstance = hInstance;
    wcex.hCursor = LoadCursor(nullptr, IDC_ARROW);
    wcex.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wcex.lpszMenuName = MAKEINTRESOURCEW(IDC_MAIN_MENU);
    wcex.lpszClassName = szWindowClass;

    RegisterClassExW(&wcex);

    HWND hWnd = CreateWindowW(szWindowClass, szTitle,
WS_OVERLAPPEDWINDOW,
        CW_USEDEFAULT, 0, 800, 600, nullptr, nullptr, hInstance,
nullptr);

    if (!hWnd) return FALSE;
```

```
    ShowWindow(hWnd, nCmdShow);
    UpdateWindow(hWnd);

    MSG msg;
    while (GetMessageW(&msg, nullptr, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
    }

    return (int)msg.wParam;
}
```

resource.h

```
#pragma once

#define IDC_MYICON              2
#define IDD_LAB2_DIALOG         102
#define IDS_APP_TITLE           103
#define IDM_ABOUT               104
#define IDM_EXIT                105
#define IDI_LAB2                107
#define IDI_SMALL               108
#define IDC_LAB2                109
#define IDC_MAIN_MENU           110

#define IDM_SHAPE_POINT         32771
#define IDM_SHAPE_LINE          32772
#define IDM_SHAPE_RECT          32773
#define IDM_SHAPE_ELLIPSE       32774
```

Resource.rc

```
#pragma code_page(65001)

#include "resource.h"
#include <windows.h>

IDC_MAIN_MENU MENU
BEGIN
    POPUP "Файл"
    BEGIN
        MENUITEM "Вихід",              IDM_EXIT
    END
    POPUP "Об'єкти"
    BEGIN
        MENUITEM "Крапка",             IDM_SHAPE_POINT
        MENUITEM "Лінія",              IDM_SHAPE_LINE
        MENUITEM "Прямокутник",        IDM_SHAPE_RECT
        MENUITEM "Еліпс",              IDM_SHAPE_ELLIPSE
    END
    POPUP "Довідка"
    BEGIN
        MENUITEM "Про програму...",    IDM_ABOUT
    END
END
```

shape.h

```
#pragma once
#include <windows.h>
#include <cmath>

class Shape {
protected:
    long xs1, ys1, xs2, ys2;
public:
    Shape() : xs1(0), ys1(0), xs2(0), ys2(0) {}
    virtual ~Shape() {}
    void Set(long x1, long y1, long x2, long y2) {
```

```
        xs1 = x1; ys1 = y1; xs2 = x2; ys2 = y2;
    }
    virtual void Show(HDC hdc) = 0;
};

// Крапка
class PointShape : public Shape {
public:
    void Show(HDC hdc) override {
        SetPixel(hdc, xs1, ys1, RGB(0, 0, 0));
        SetPixel(hdc, xs1 + 1, ys1, RGB(0, 0, 0));
        SetPixel(hdc, xs1 - 1, ys1, RGB(0, 0, 0));
        SetPixel(hdc, xs1, ys1 + 1, RGB(0, 0, 0));
        SetPixel(hdc, xs1, ys1 - 1, RGB(0, 0, 0));
    }
};
//лінія
class LineShape : public Shape {
public:
    void Show(HDC hdc) override {
        HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
        HGDIOBJ oldPen = SelectObject(hdc, hPen);

        MoveToEx(hdc, xs1, ys1, NULL);
        LineTo(hdc, xs2, ys2);

        SelectObject(hdc, oldPen);
        DeleteObject(hPen);
    }
};

// Прямокутник
class RectShape : public Shape {
public:
    void Show(HDC hdc) override {
        HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
        HBRUSH hBrush = (HBRUSH)GetStockObject(NULL_BRUSH);

        HGDIOBJ oldPen = SelectObject(hdc, hPen);
        HGDIOBJ oldBrush = SelectObject(hdc, hBrush);

        long dx = std::abs(xs2 - xs1);
        long dy = std::abs(ys2 - ys1);
```

```
        Rectangle(hdc, xs1 - dx, ys1 - dy, xs1 + dx, ys1 + dy);

        SelectObject(hdc, oldPen);
        SelectObject(hdc, oldBrush);
        DeleteObject(hPen);
    }
};

// Еліпс
class EllipseShape : public Shape {
public:
    void Show(HDC hdc) override {
        HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
        HBRUSH hBrush = CreateSolidBrush(RGB(255, 165, 0));

        HGDIOBJ oldPen = SelectObject(hdc, hPen);
        HGDIOBJ oldBrush = SelectObject(hdc, hBrush);

        Ellipse(hdc, xs1, ys1, xs2, ys2);

        SelectObject(hdc, oldPen);
        SelectObject(hdc, oldBrush);
        DeleteObject(hPen);
        DeleteObject(hBrush);
    }
};
```